

Rapport Final de Projet

Gestion de Stock et Validation Automatisée

Youssef Ben Younes & Mohamed Messaoudi

22 janvier 2026

Table des matières

1	Introduction et Analyse	2
1.1	Introduction	2
1.2	Objectifs du Projet	2
1.3	Analyse des Besoins	2
1.3.1	Besoins Fonctionnels	2
1.3.2	Diagrammes de Cas d'Utilisation	2
2	Conception et Implémentation	4
2.1	Outils et Architecture	4
2.1.1	Liste des Outils Utilisés	4
2.1.2	Diagramme d'Architecture et Outils	4
2.2	Modélisation Métier	4
2.2.1	Diagramme de Classes	4
2.2.2	Diagramme de Séquence (Commande)	5
3	Validation et Assurance Qualité	7
3.1	Stratégie de Test	7
3.2	Scénarios de Test Exécutés	7
3.3	Résultats des Tests	8
4	Manuel d'Utilisation et Conclusion	9
4.1	Manuel d'Utilisation Simplifié	9
4.2	Conclusion	9

Chapitre 1

Introduction et Analyse

1.1 Introduction

Ce projet a pour objectif de concevoir et développer une application de gestion de stock en langage Python. Cette application permet de gérer les produits et les commandes d'une entreprise, en assurant la persistance des données via JSON et base de données MySQL, ainsi que la validation de la qualité par des tests automatisés.

1.2 Objectifs du Projet

Les principaux objectifs sont :

- Valider les compétences acquises en programmation Python et MySQL.
- Mettre en œuvre les concepts de la Programmation Orientée Objet (POO).
- Créer une interface utilisateur graphique (GUI) moderne avec PyQt6.
- Assurer la robustesse de l'application via des tests automatisés (Pytest).

1.3 Analyse des Besoins

1.3.1 Besoins Fonctionnels

L'application permet de :

- **Gestion des Produits** : Ajout, modification, archivage, affichage trié.
- **Gestion des Commandes** : Cycle complet (Brouillon → Confirmé → Payé → Livré).
- **Persistance** : Synchronisation bidirectionnelle entre fichiers JSON locaux et base de données MySQL.
- **Statistiques** : Tableau de bord des ventes et produits populaires (Revenu total, Top produits).

1.3.2 Diagrammes de Cas d'Utilisation

Le diagramme suivant détaille la gestion des produits, les statistiques et les commandes :

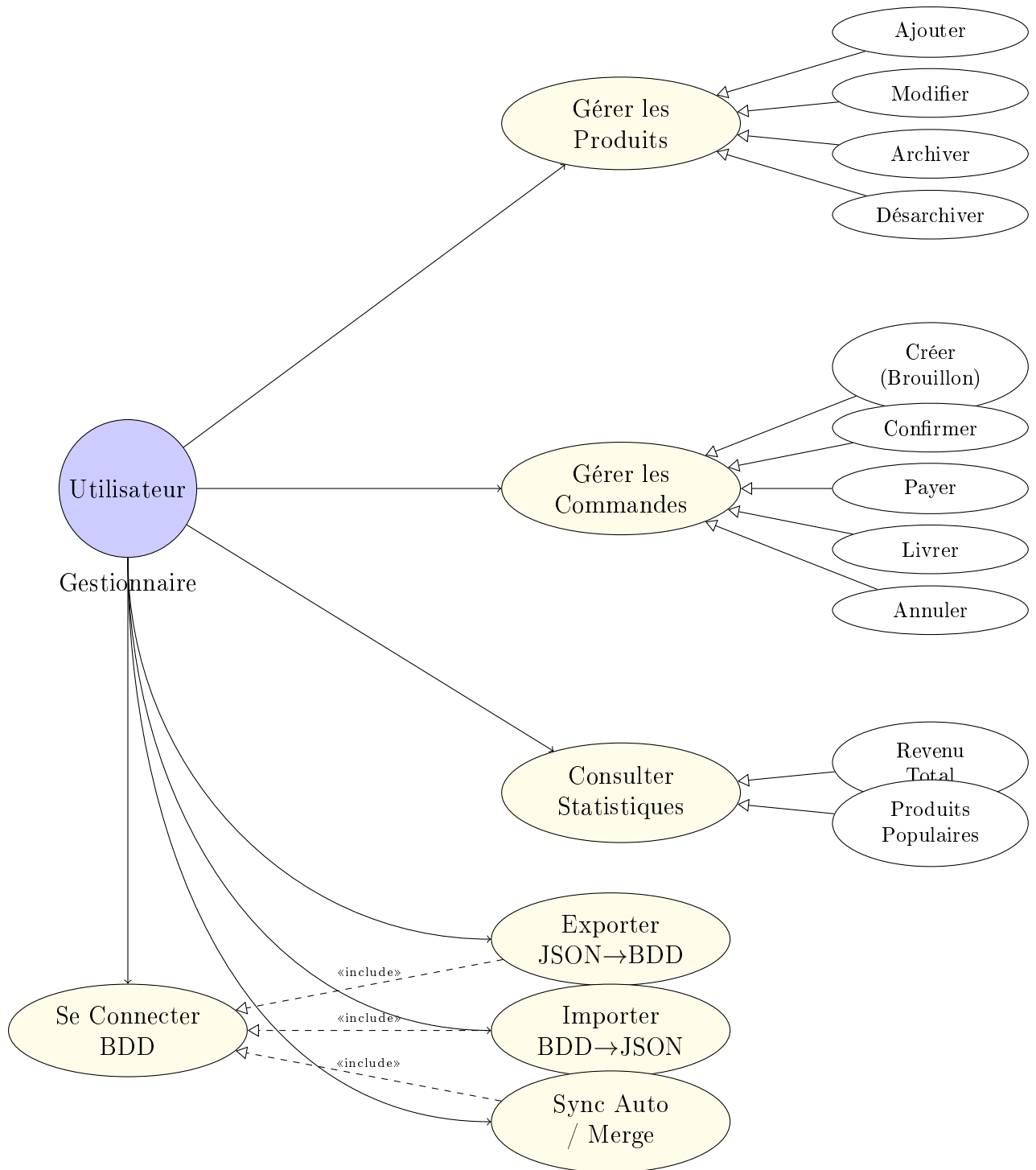


FIGURE 1.1 – Diagramme de Cas d'Utilisation Détaillé

Chapitre 2

Conception et Implémentation

2.1 Outils et Architecture

2.1.1 Liste des Outils Utilisés

Pour mener à bien ce projet, nous avons utilisé un ensemble d'outils modernes :

- **Python 3.12** : Langage de programmation principal.
- **PyQt6** : Bibliothèque pour l'interface graphique.
- **MySQL Connector** : Pilote pour la connexion à la base de données.
- **Pytest & Pytest-Qt** : Frameworks pour les tests unitaires et UI.
- **Git** : Gestion de version.
- **LaTeX** : Rédaction de la documentation technique.

2.1.2 Diagramme d'Architecture et Outils

L'architecture globale du système intègre ces outils comme suit :

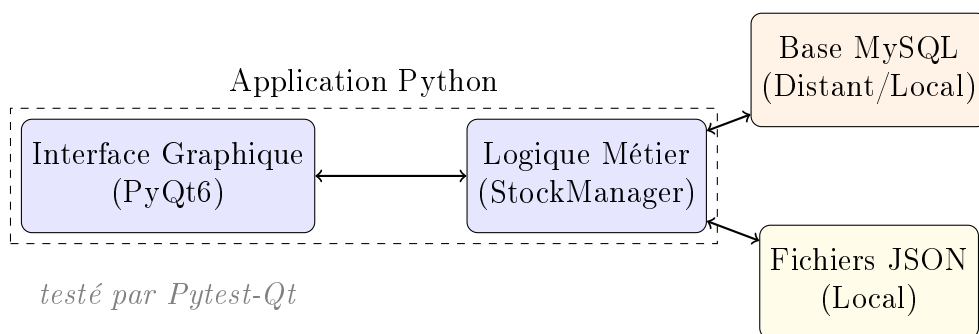


FIGURE 2.1 – Diagramme Outils et Architecture

2.2 Modélisation Métier

2.2.1 Diagramme de Classes

Le diagramme ci-dessous représente la structure des classes principales :

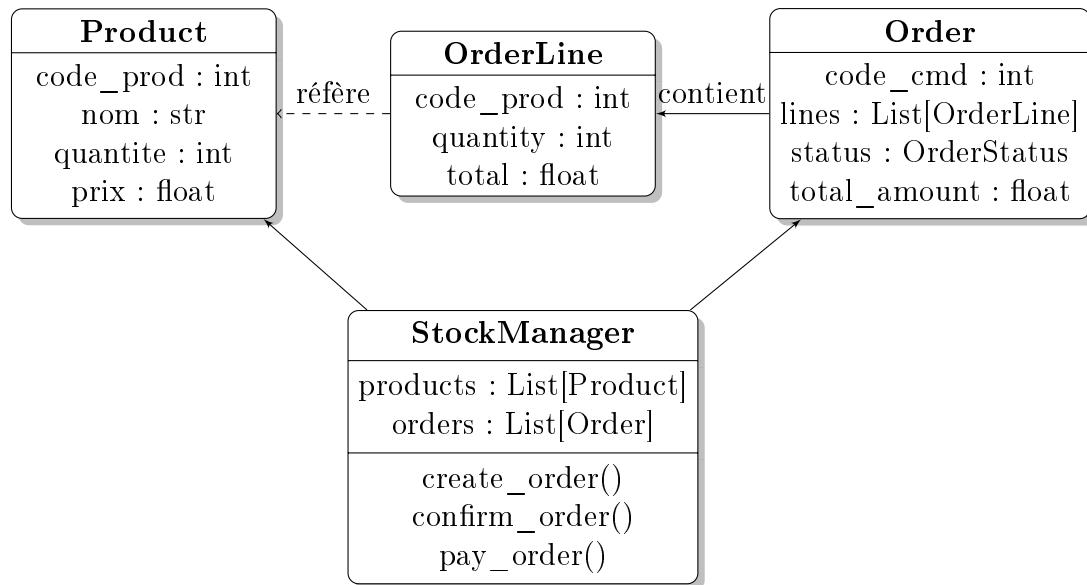


FIGURE 2.2 – Diagramme de Classes Simplifié

2.2.2 Diagramme de Séquence (Commande)

Ce diagramme détaille le traitement d'une commande, incluant la confirmation (ou annulation), le paiement et la livraison.

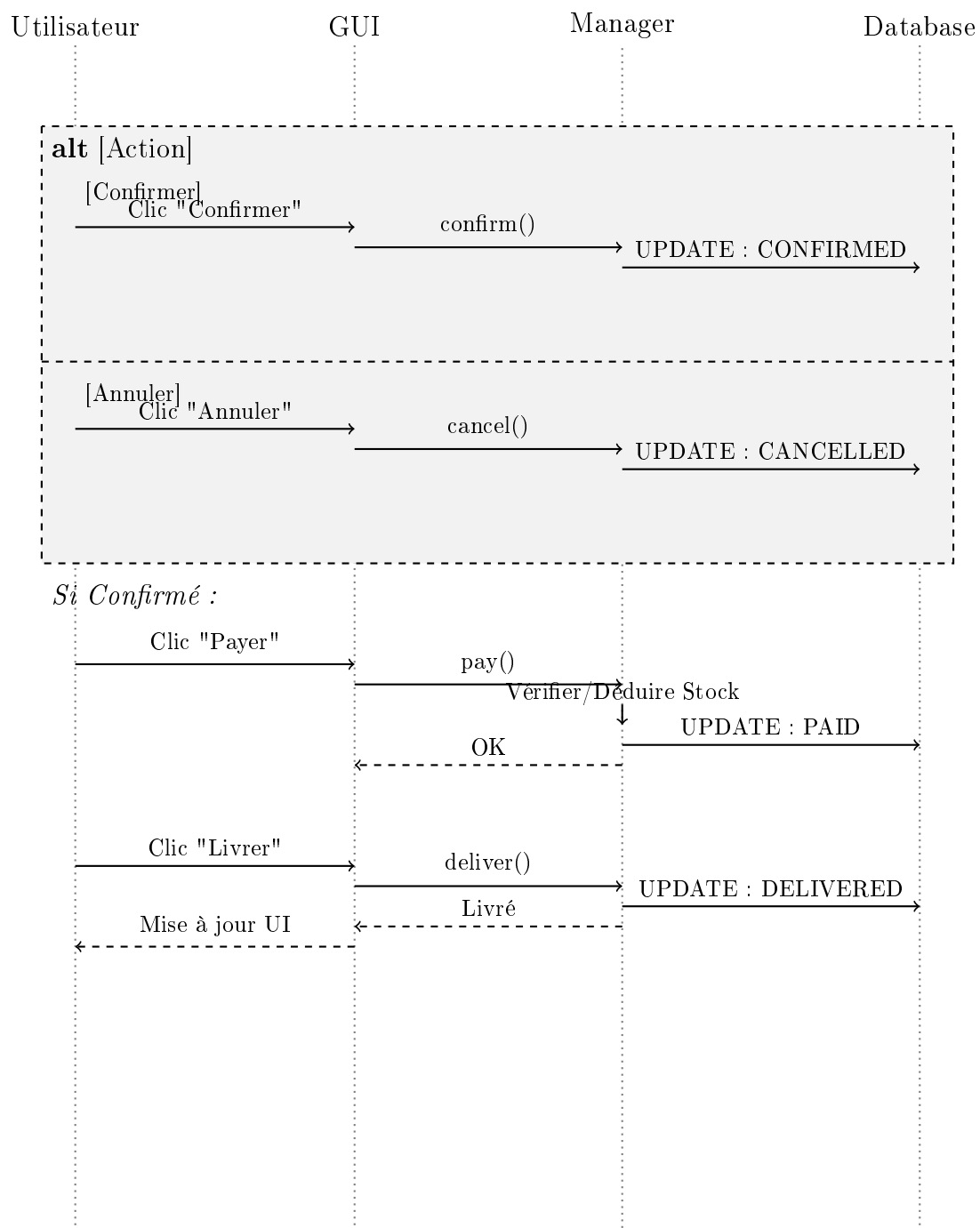


FIGURE 2.3 – Diagramme de Séquence : Cycle de Vie Commande

Chapitre 3

Validation et Assurance Qualité

3.1 Stratégie de Test

Pour garantir la fiabilité de l'application suite aux nombreux changements (notamment l'introduction de MySQL et la refonte UI), nous avons mis en place une stratégie de tests automatisés utilisant :

- **Pytest** : Framework de test standard pour Python.
- **Pytest-Qt** : Plugin permettant de simuler des interactions utilisateur (clics, saisies) sur l'interface PyQt6 sans intervention humaine.

Cette approche permet de détecter rapidement les régressions lors du développement.

3.2 Scénarios de Test Exécutés

Les tests suivants ont été implémentés dans `tests/test_ui.py` :

ID Test	Description du Scénario	Justification
UI-01	Ajout de Produit (Cas Nominal)	Vérifie que la chaîne complète (Saisie → Manager → UI Table) fonctionne correctement.
UI-02	Validation de Saisie	Assure que l'utilisateur ne peut pas corrompre les données (ex : produit sans nom).
UI-03	Cycle de Vie Commande	Valide le flux métier complexe : création, ajout de lignes, confirmation et paiement avec déduction de stock.

ID Test	Description du Scénario	Justification
UI-04	Archivage	Vérifie la fonctionnalité de suppression logique (soft-delete) essentielle pour l'historique.

3.3 Résultats des Tests

Les tests automatisés ont été exécutés avec succès, confirmant la stabilité de la version actuelle.

Scénario	Résultat	Temps Exécution
UI-01 Ajout Produit	SUCCÈS	0.15s
UI-02 Validation	SUCCÈS	0.05s
UI-03 Commandes	SUCCÈS	0.42s
UI-04 Archivage	SUCCÈS	0.10s

Chapitre 4

Manuel d'Utilisation et Conclusion

4.1 Manuel d'Utilisation Simplifié

1. Lancer l'application : `python main.py`
2. **Onglet Accueil** : Configurer la connexion MySQL si nécessaire.
3. **Onglet Produits** : Ajouter le stock initial.
4. **Onglet Commandes** : Créer une commande ("Brouillon"), ajouter des lignes, puis "Confirmer" et "Payer".
5. **Onglet Statistiques** : Visualiser les performances (Top produits, Revenu total) via des graphiques interactifs.

4.2 Conclusion

Ce projet a permis d'intégrer des technologies professionnelles (Base de données relationnelle, Tests UI Automatisés) dans une application Python concrète. La mise en place des tests automatisés (`pytest-qt`) apporte une garantie de qualité logicielle, facilitant les futures évolutions et la maintenance du code.